



CENTRO DE TECNOLOGIA – CTC

DEPARTAMENTO DE INFORMÁTICA – DIN

BACHARELADO EM INFORMÁTICA

NAYANE BATISTA COSTA

YURI PIRES ALVES

**RELATÓRIO TÉCNICO EM COMPILADORES: Implementação das Análises Léxica,
Sintática e Semântica com *Lex* e *Yacc***

Maringá,

2024

Sumário

1. Facilidades e Dificuldades	2
2. Processo de Construção	3
2.1. <i>Tokens</i>	4
2.2. Regras da Gramática e Tabela de Símbolos	6
2.3. Árvore Sintática	8
2.4. Observações Adicionais	10
3. Rodando o projeto	10
4. Referências e <i>Links</i> Utilizados	10

1. Facilidades e Dificuldades

Inicialmente, evidenciamos que o uso das *lib Flex* e *yacc/Bison* facilitou significativamente o processo de implementação, devido às funcionalidades úteis já prontas que elas fornecem. No que diz respeito à etapa de pesquisa da documentação e exemplos disponíveis, principalmente via tutoriais do *YouTube*, alguns materiais não elucidaram completamente a aplicação das ferramentas para construção de um compilador próprio, tornando o primeiro contato difícil devido a especificidade.

Já para as dificuldades, o comando “*-ll (link in the lex library)*” encontrado em grande parte dos materiais não funcionou com nosso programa quando a análise léxica foi implementada e, nas documentações não havia informações que bastava retirá-lo do terminal que o código rodaria sem restrições, atrasando o trabalho da equipe até que o caso fosse solucionado. Paralelo a esse erro, houve ocorrências de avisos de conflitos de deslocamento na gramática, que exigiu revisão das regras do *parser.y*, no entanto, não ficou clara as razões para essas acusações de conflitos mesmo com o uso do *-Wcounter* e esses avisos não prejudicavam o funcionamento do compilador, então não foram solucionados.

Além disso, o *yacc* possui a funcionalidade *yyerror* que tem que ser declarada, caso contrário, ele te impede de rodar a implementação. O empecilho aqui é que isso te impede de especificar se os erros são léxicos, pois o *yyerror* assume que todo erro lido por ele é sintático e exibe no *prompt* essa informação. Mesmo com isso, conseguimos printar o que estaria errado e exibir a linha da ocorrência do erro, mas com a restrição de “*syntax error*” dada pelo próprio *yyerror*.

Na etapa da construção da árvore sintática, houve um trabalho manual significativo para declaração dos nós, contudo encontramos uma alternativa de impressão da mesma em um arquivo no formato *JSON* e isso facilitou a visualização para verificação da acurácia das informações exibidas pela árvore gerada. Porém, houve também durante a implementação dos vetores e das funções, muitas ocorrências do “*error: has no declared type*” ao tentarmos atribuir os *nodes* nessas funcionalidades.

Por fim, na análise semântica, por ser o último estágio do desenvolvimento prático, o código-fonte do compilador já estava “crescido” numa proporção considerável, então se tornou difícil e complicado ficar navegando por ele buscando retomar as regras gramaticais no *parser.y*, que somava mais de 750 linhas, ou outros tipos de informações para manipulá-las em blocos distintos do código que seriam destinados às verificações da semântica.

2. Processo de Construção

Resumidamente, a análise léxica é a primeira fase, responsável por especificar os *tokens* individuais na entrada do programa e, o próximo passo é a análise sintática, que usa uma gramática pré-definida para verificar a estrutura do programa. Executamos por conseguinte, ações semânticas associadas a cada regra da gramática, verificando e garantindo que o código está de acordo com elas.

A princípio, nosso compilador lerá uma linguagem que chamaremos de *.uai* que possui uma formatação parecida com o C e aproveita conceitos em comum, porém sua marca registrada é o fato de que ela foi elaborada com palavras-chave em português do Brasil. Como a linguagem C, a *.uai* possui tipagem estática, ou seja, as variáveis precisam ter seus tipos declarados, os comandos devem ser finalizados com “;” e os blocos devem ser delimitados com um par de chaves “{ }”.

Para a composição da análise sintática, foi declarada uma *struct* que representa o nó da árvore abstrata a ser gerada. O nó da *struct* tem os atributos *left*, *right* e *token*. Para gerar a árvore sintática, é criado um nó para cada *token* e esse é ligado aos nós dos *tokens* subsequentes que ocorrem à sua esquerda e à sua direita. A leitura *in-order* deve replicar logicamente o programa lido.

Nosso analisador semântico foi planejado para tratar três tipos de validações:

- **Verificação de declaração prévia** → as variáveis devem ser declaradas antes de serem utilizadas. Para isso, usamos a função *check_declaration* que verifica se o *char* identificador passado como parâmetro está presente na tabela de símbolos. Se não estiver, é impressa uma mensagem de erro informando que a variável não foi declarada anteriormente.
- **Checagem de tipo** → refere-se à verificação dos tipos das variáveis em uma expressão. Para isso, é utilizada a função *check_types*, que recebe os tipos de ambas as variáveis a serem comparadas como entrada. Se os tipos coincidirem e forem compatíveis, nada é feito.
- **Uso correto das palavras reservadas** → é feita uma comparação para confirmar se a variável declarada pertence a lista de palavras-chave reservadas ["inteiro", "flutuante", "caractere", "vazio", "se", "senao", "para", "principal", "retorno", "incluir"]. Isso é executado antes de adicioná-la à tabela de símbolos durante a declaração.

2.1. Tokens

Definimos palavras-chave como: imprimir, ler, inteiro, vazio; identificadores formados por $(\{\alpha\}(\{\alpha\}|\{\text{digit}\})^*)$, operadores matemáticos como: -, +, /, *; operadores relacionais como: >, <, <=, >=, ==, !=; laço de repetição “para” e comentários.

Figura 1 – *lexer.l* com os *tokens* da *uai*.

```
"principal"           { return MAIN; }
"imprimir"            { return PRINTF; }
"ler"                 { return SCANF; }
"inteiro"             { return INT; }
"flutuante"           { return FLOAT; }
"caractere"           { return CHAR; }
"vazio"               { return VOID; }
"retornar"            { return RETURN; }
"para"                { return FOR; }
"se"                  { return IF; }
"senao"               { return ELSE; }
^"#incluir"[ ]*<.\.h> { return INCLUDE; }
"verdadeiro"          { return TRUE; }
"falso"               { return FALSE; }
[-]?{digit}+          { return NUMBER; }
[-]?{digit}+\.{digit}{1,6} { return FLOAT_NUMBER; }
{alpha}({alpha}|{digit})* { return ID; }
{unary}               { return UNARY; }
"<="                  { return LE; }
">="                  { return GE; }
"=="                  { return EQ; }
"!="                  { return NE; }
">"                  { return GT; }
"<"                  { return LT; }
"+"                   { return ADD; }
"-"                   { return SUBTRACT; }
"/"                   { return DIVIDE; }
"*"                   { return MULTIPLY; }
\\\/\.*               { ; }
\\\/\*(.*\n)*.*\*\\\/ { ; }
[ \t]*                { ; }
[\n]                  { countn++; }
.                      { return *yytext; }
["].*["               { return STRING; }
[''].*['']            { return CHARACTER; }
```

Figura 2 – Exemplo parcial dos *tokens* ao serem feitos impressos.

```
./a.out<input.uai
Token: VOID      Valor: vazio
Token: ID        Valor: soma
Token: SYMBOL    Valor: (
Token: SYMBOL    Valor: )
Token: SYMBOL    Valor: {
Token: PRINTF    Valor: imprimir
Token: SYMBOL    Valor: (
Token: STRING    Valor: "salve"
Token: SYMBOL    Valor: )
Token: SYMBOL    Valor: ;
Token: SYMBOL    Valor: }
Token: INT       Valor: inteiro
Token: MAIN      Valor: principal
Token: SYMBOL    Valor: (
Token: SYMBOL    Valor: )
Token: SYMBOL    Valor: {
Token: ID        Valor: soma
Token: SYMBOL    Valor: (
Token: SYMBOL    Valor: )
Token: SYMBOL    Valor: ;
Token: INT       Valor: inteiro
Token: ID        Valor: a
Token: SYMBOL    Valor: ;
Token: INT       Valor: inteiro
Token: ID        Valor: x
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 1
Token: SYMBOL    Valor: ;
Token: INT       Valor: inteiro
Token: ID        Valor: y
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 2
Token: SYMBOL    Valor: ;
Token: INT       Valor: inteiro
Token: ID        Valor: z
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 3
Token: SYMBOL    Valor: ;
Token: ID        Valor: x
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 3
Token: SYMBOL    Valor: ;
Token: ID        Valor: y
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 10
Token: SYMBOL    Valor: ;
Token: ID        Valor: z
Token: SYMBOL    Valor: =
Token: NUMBER    Valor: 5
Token: SYMBOL    Valor: ;
Token: IF        Valor: se
Token: SYMBOL    Valor: (
Token: ID        Valor: x
```

2.2. Regras da Gramática e Tabela de Símbolos

1. *<program>*: Consiste em *headers* opcionais e uma lista de funções.
2. *<headers>*: Define os *headers* incluídos no programa, podendo ser eles opcionais (vazio) ou um ou mais *headers*, que serão adicionados à tabela de símbolos.
3. *<main>*: Define função principal, começa com a declaração seguida pelo *body* e finaliza com um *return*.
4. *<main_declaration>*: Declara a função com a palavra-chave “principal” seguida por parênteses vazios que indicam que ela não recebe parâmetros.
5. *<function_list>*: Lista de funções do programa, podendo ser uma única *function_declaration* ou uma sequência delas.
6. *<function_declaration>*: Define uma função que começa com o nome, seguido por um bloco entre chaves e finalizando com o retorno. Pode ser também a principal.
7. *<function_name>*: Define o nome da função (ID) e seus parâmetros opcionais que estarão dentro de parênteses.
8. *<parameter_list>*: Define os parâmetros de uma função. Quando este não é vazio, ou seja, opcional, seus parâmetros devem estar separados por vírgula.
9. *<function_call>*: Chamada de uma função que pode vir a ter argumentos.
10. *<arguments_list>*: Argumentos de uma chamada de função, pode ser vazio ou ter mais de um valor, sendo estes separados por vírgulas.
11. *<datatype>*: Define os tipos de dados da linguagem como inteiro, flutuante, caractere e vazio (INT, FLOAT, CHAR, VOID).
12. *<body>*: Define o corpo da função, contendo os comandos que podem ser contidos, como laços “para”, estruturas condicionais “se” e “senao”, chamadas de funções, declarações, outro *body* (blocos de código), comandos de entrada “ler” e saída “imprimir”.
13. *<else>*: A cláusula “senao” é opcional e segue a estrutura condicional “se”.
14. *<condition>*: Definem condições que podem ser expressões que usam operadores relacionais para comparar valores ou *booleanos* “verdadeiro” e “falso”.
15. *<array_declaration>*: Declara um vetor com um ID que o representará e o tamanho deste *array*.
16. *<statement>*: Define declaração de variáveis, atribuições e comparações.
17. *<init>*: Define a inicialização de uma variável durante a declaração.

18. *<expression>*: Define expressões matemáticas e lógicas usando operadores aritméticos entre expressões ou um valor único.
19. *<arithmetic>*: Define os operadores aritméticos soma (+), subtração (-), multiplicação (*) e divisão (/).
20. *<relop>*: Define os operadores relacionais menor que (<), maior que (>), menor ou igual a (<=), maior ou igual a (>=), igual a (==) e diferente de (!=).
21. *<value>*: Valores que podem ser usados em expressões como números inteiros, números de ponto flutuante, caracteres, representando constantes numéricas e IDs, representando variáveis. Rege também o acesso a um elemento de um *array* utilizando colchetes que contém uma expressão.
22. *<return>*: Retorno da função que pode ser vazio, indicando que a função não retorna um valor.

Para construção da tabela de símbolos (*symbolTable*), ao encontrarmos um identificador (ID), chama-se a função *add()* para adicionar informações relevantes à tabela de símbolos, enquanto a função *search()* verifica se um determinado identificador já está presente, pois se estiver, um erro semântico vai ser registrado.

Se o ID não for duplicado e não for uma palavra reservada da *.uai*, uma nova entrada é adicionada na tabela, contendo o nome do identificador (*id_name*), o tipo de dado (*data_type*), a categoria (*type*) e o número da linha onde foi encontrado (*line_no*).

Os contextos passados para *add()* são definidos como:

- "**Header**" → Representa um arquivo de cabeçalho incluído no código.
- "**Keyword**" → O ID é uma palavra reservada da linguagem.
- "**Variable**" → O ID representa uma variável.
- "**Constant**" → O ID representa uma constante.
- "**Function**" → O ID representa uma função.
- "**ArraY**" → O ID representa um *array*.

Um exemplo de uma tabela de símbolos exibida pelo nosso compilador pode ser visto abaixo.

Figura 3 – Configuração da tabela de símbolos.

SÍMBOLO	TIPO DO DADO	CATEGORIA	LINHA
#incluir<stdio.h>		Header	1
#incluir<string.h>		Header	2
principal	inteiro	Function	4
a	inteiro	Variável	5
x	inteiro	Variável	6
1	CONST	Constante	6
y	inteiro	Variável	7
2	CONST	Constante	7
z	inteiro	Variável	8
3	CONST	Constante	8
10	CONST	Constante	10
5	CONST	Constante	11
se	N/A	Palavra-chave	12
para	N/A	Palavra-chave	13
k	inteiro	Variável	13
0	CONST	Constante	13
imprimir	N/A	Palavra-chave	15
senao	N/A	Palavra-chave	17
idx	inteiro	Variável	18
i	inteiro	Variável	20
ler	N/A	Palavra-chave	22
j	inteiro	Variável	26
retornar	N/A	Palavra-chave	30

2.3. Árvore Sintática

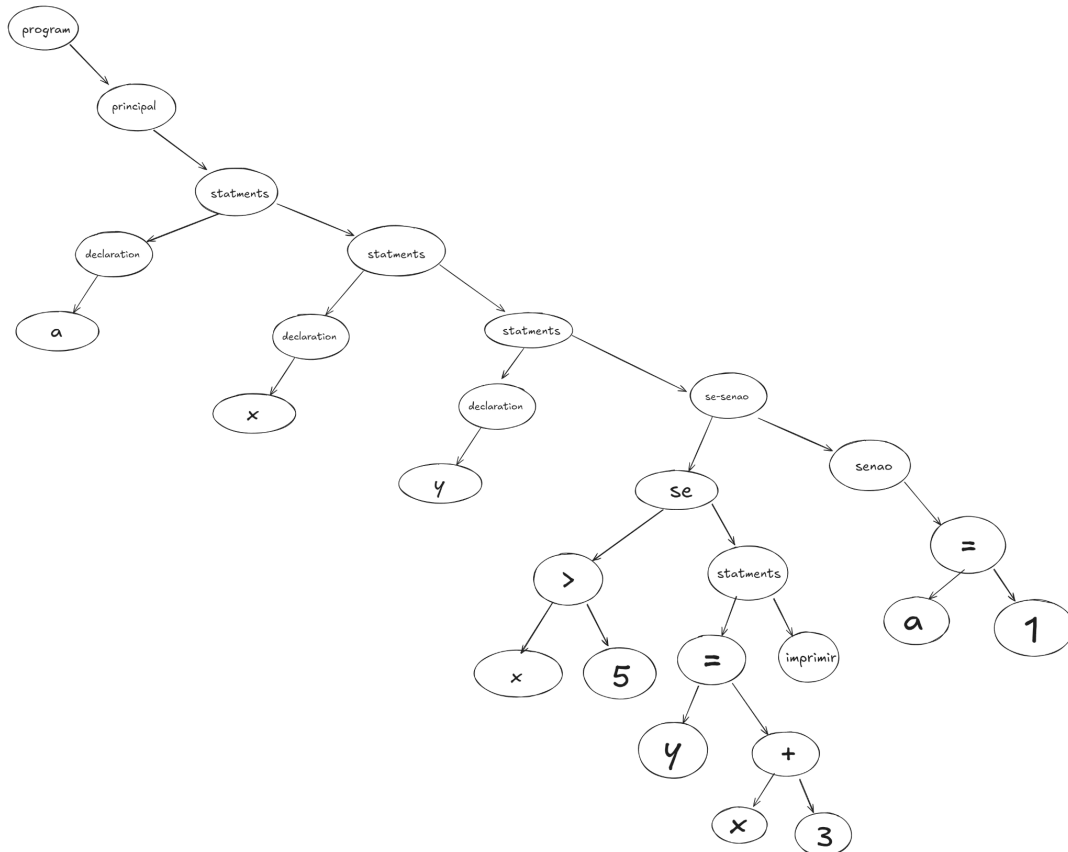
Figura 4 – Exemplo de uma árvore sintática desenhada a partir dos dados do *JSON*.

Figura 5 – Trecho parcial de um arquivo *JSON*.

```

1  {
2    "token": "program",
3    "left": null,
4    "right": {
5      "token": "principal",
6      "left": null,
7      "right": {
8        "token": "statements",
9        "left": {
10         "token": "declaration",
11         "left": {
12           "token": "a",
13           "left": null,
14           "right": null
15         },
16         "right": {
17           "token": "NULL",
18           "left": null,
19           "right": null
20         }
21       },
22       "right": {
23         "token": "statements",
24         "left": {
25           "token": "declaration",
26           "left": {
27             "token": "x",
28             "left": null,
29             "right": null
30           },
31           "right": {
32             "token": "NULL",
33             "left": null,
34             "right": null
35           }
36         },
37         "right": {
38           "token": "statements",
39           "left": {
40             "token": "declaration",
41             "left": {
42               "token": "y",
43               "left": null,
44               "right": null
45             },
46             "right": {
47               "token": "NULL",
48               "left": null,
49               "right": null
50             }
51           },

```

2.4. Observações Adicionais

Caso seja identificado um erro léxico ou sintático, o programa encerra a leitura logo na impressão da tabela de *tokens*, não seguindo para as outras fases de compilação.

3. Rodando o projeto

Primeiramente, precisamos das ferramentas instaladas em nosso ambiente de desenvolvimento, nesse caso, o sistema operacional usado foi o *Linux Ubuntu 22.04.4 LTS*. Os passos para obter as bibliotecas foram:

```
sudo apt-get install flex  
sudo apt-get install bison
```

Posterior a isso, para rodarmos a aplicação no terminal, usamos o integrado ao ambiente de desenvolvimento *Visual Studio Code*, é necessário seguir os passos abaixo:

```
lex lexer.l  
yacc -d parser.y  
gcc -w y.tab.c  
./a.out<input.uai
```

Ou, para automatizar o processo, basta inserir o seguinte comando:

```
make
```

Para realizar os testes adicionais posteriores do programa, não precisa compilar novamente o *lexer.l* e o *parser.y*, basta modificar apenas o arquivo *input.uai*, salvar e chamá-lo novamente no terminal com o comando:

```
./a.out<input.uai
```

4. Referências e *Links* Utilizados

Para construção e garantia do funcionamento do nosso compilador, foram consultados os materiais elencados nesta seção.

<https://cs.lynchburg.edu/briggs/classes/322/notes/lexYacc/how_to_write_and_run_programs.htm>

<<https://cse.iitkgp.ac.in/~bivasm/notes/LexAndYaccTutorial.pdf>>

<<https://efxa.org/2014/05/25/how-to-create-an-abstract-syntax-tree-while-parsing-an-input-stream/>>

<https://ftp.gnu.org/old-gnu/Manuals/flex-2.5.4/html_mono/flex.html>

<<https://lambda.uta.edu/cse5317/notes/node26.html>>

<<https://medium.com/codex/building-a-c-compiler-using-lex-and-yacc-446262056aaa>>

<<https://pgrandinetti.github.io/compilers/page/implementation-semantic-analysis/>>

<<https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/180%20Semantic%20Analysis.pdf>>

<https://www.youtube.com/watch?v=2hNqBq8HQkw&list=PLImMVrOC3KFn0US0AiW0JYLaU8mCtrqG0&ab_channel=ShreyasNisal>

<https://www.youtube.com/watch?v=AxLgkLIAOCc&list=PL0xf_aizy1d45ufPr82TGyEVyzTmTmsO4&index=1&ab_channel=ComputerHub>

<<https://zarif98sjs.github.io/minecraft/Yet-Another-C-Compiler/>>